Vultr Agent Rush Hackathon: Complete Technical Guide, Architecture Playbook, and Live Event Intelligence

The URL https://lablab.build/ai-hackathons/vultr-hackathon is the official competition portal for the **Vultr: Agent Rush Hackathon**, a five-day hybrid AI engineering competition co-organized by **LabLab** (operated by NativelyAI Inc.) and **Vultr**[1]. Running online from **November 3 to November 8, 2026**, and culminating in an in-person finale on **November 8, 2026**, at **Pierpont Place** in Salt Lake City, Utah, the event challenges solo builders and teams of up to six members to build production-grade, web-accessible AI agents hosted on Vultr Cloud Compute[1]. Unlike conversational wrapper hackathons, the central mandate of Agent Rush is **containment-first execution**: building autonomous systems where a Vultr virtual machine (VM) acts as the central control and orchestration plane, dispatching generated code, browser automation, or multi-step enterprise workflows into isolated execution environments[1].

Participants compete across two distinct tracks—**Track 01: Blast Radius Zero** (focused on sandboxed code execution and browser automation with strict kernel or container isolation) and **Track 02: Powering the Future of Work** (focused on multi-step enterprise workflow orchestration across sales, finance, HR, support, and procurement)—for a total prize and giveaway pool of **up to $15,000**[1]. The competition prize pool totals **$14,000** (**$9,000 in cash** and **$5,000 in Vultr credits** split across first, second, and third place), supplemented by an on-site hardware raffle in Salt Lake City valued at up to **$1,000** (featuring a PlayStation 5 Standard, Apple AirPods 5 with Wireless Charging Case, and a Razer DeathAdder V3 HyperSpeed mouse)[1]. Additionally, every registered participant is eligible to claim **$200 in free Vultr cloud credits** to provision compute instances, managed databases, Kubernetes clusters, and serverless inference during the build window[1].

As of the latest live competition telemetry during the November 3–8 build window, the hackathon has scaled to **2,536 registered participants**, **714 teams**, **50 completed project submissions**, and **20 active drafts in progress**[6]. Prospective entrants and active builders must account for four critical operational constraints documented by the organizers: (1) all official project entries **must** be submitted on lablab.ai under an original, MIT-compliant open-source license to qualify for judging (sharing on the Vultr Community portal is optional for extra visibility and does not replace the LabLab submission); (2) **Vultr Cloud GPUs are not available** for this hackathon, requiring teams to rely on Vultr Cloud Compute VMs and the OpenAI-compatible **Vultr Serverless Inference API** (https://api.vultrinference.com/v1); (3) attendance at the Salt Lake City on-site showcase is by approval only, with travel and accommodation expenses not covered by the organizers; and (4) official prize distribution may take **up to 90 days** post-event[1].

LABLAB × VULTR · HYBRID AI HACKATHON (NOV 3–8, 2026)

● LIVE TELEMETRY · SUBMISSIONS OPEN

Vultr Agent Rush Surpasses 2,500 Builders as Teams Converge on Vultr Compute and Qwen Models

1. LIVE PARTICIPATION & VELOCITY TELEMETRY

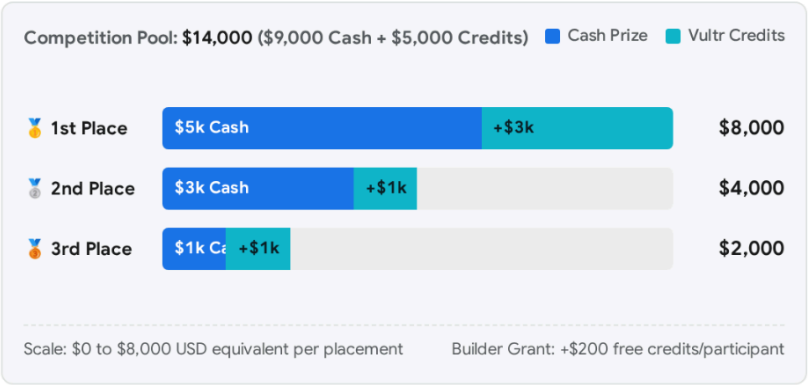
Showing Latest Dashboard Snapshot vs. Earlier Intraday Baseline

Latest Snapshot | Earlier Snapshot

Participants	Teams	Submissions	Drafts in Progress	Participant Credit Grant
2,536	714	50	20	$200
▲ +97 today	▲ +50 today	▲ +29 today	▲ +7 today	Per builder
+96 vs. earlier snapshot	+44 vs. earlier snapshot	+24 vs. earlier snapshot	+6 vs. earlier snapshot	Free Vultr infra credits

2. PRIZE STRUCTURE & INCENTIVE ALLOCATION (UP TO $15,000 TOTAL)

Cash vs. Vultr Cloud Credits by placement, plus Salt Lake City finale hardware giveaway



UP TO $1,000 VALUE

Onsite Hardware Raffle

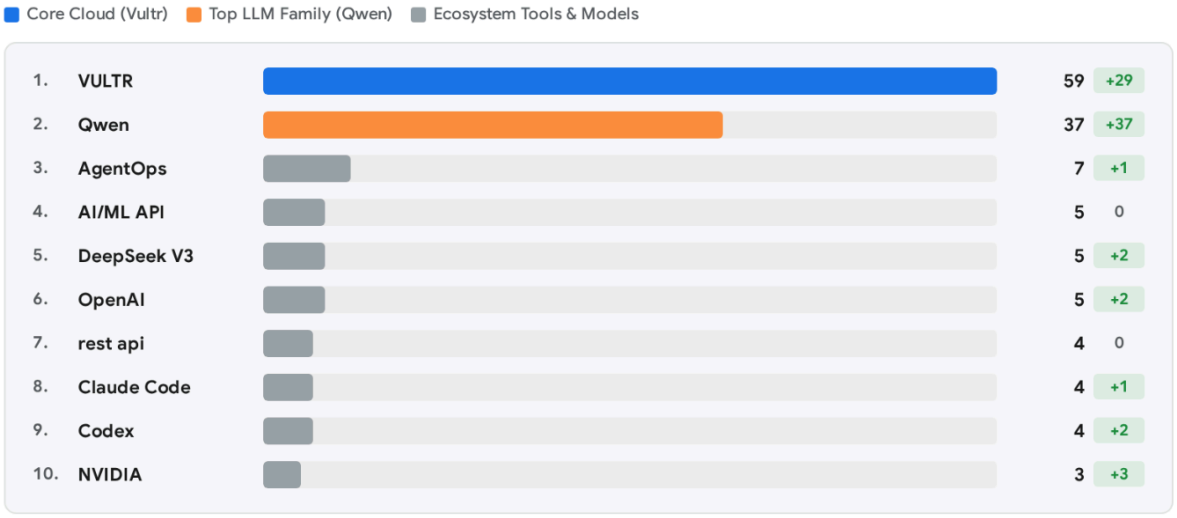
📍 Pierpont Place · Salt Lake City, UT (Nov 8, 2026)

- PlayStation 5 Standard
- Apple AirPods 5 with Wireless Charging Case
- Razer DeathAdder V3 HyperSpeed Wireless Mouse

3 individual onsite attendees win 1 hardware item each.

3. TOP 10 TECHNOLOGIES & LLM FAMILIES IN ACTIVE SUBMISSIONS

Ranked by active team tags across 20 total tracked ecosystem tools



Live event telemetry reflects the latest dashboard snapshot during the November 3–8, 2026 build window, showing **+97 new participants** and **+29 new submissions** recorded in a single 24-hour cycle. Beyond the $14,000 competition pool, onsite attendees in Salt Lake City compete in a hardware raffle worth up to $1,000, and every registered builder receives $200 in Vultr infrastructure credits.

Data sources: LabLab Vultr: Agent Rush Hackathon · LabLab Live Telemetry Dashboard · Vultr Community Event Page

Event Logistics, Timeline, and Prize Economics

The **Vultr: Agent Rush Hackathon** is structured as a five-day global build sprint that bridges remote cloud engineering with an in-person industry showcase[1]. The online build phase runs from **Tuesday, November 3, 2026, through Sunday, November 8, 2026**, welcoming builders across all experience levels and geographies[1]. The culminating on-site finale takes place on **Sunday, November 8, 2026**, at **Pierpont Place** (163 W Pierpont Ave, Salt Lake City, UT 84101, USA)[1]. The choice of Salt Lake City and the November 8 date aligns with the broader cloud-native and AI infrastructure calendar in Utah: Salt Lake City hosts several Kubernetes and AI systems gatherings immediately following the hackathon finale—including *Agent Substrate Day*, *GKE Day @ KubeCon SLC*, and *Kubernetes Automation Day* starting Monday, November 9, 2026[8].

Because the event is coordinated across multiple community platforms—including lablab.build / lablab.ai, the **Vultr Community** portal (community.vultr.com), **Luma** (luma.com/lablab.ai), and regional hubs such as **Silicon Slopes** and **Cerebral Valley**—participants will encounter slightly different timestamp conventions depending on which portal they consult[1]. Reconciling these schedules is essential for avoiding missed check-ins or submission cutoffs:

Event Phase / Milestone	Official Date	Published Time Window	Platform / Venue & Operational Notes
Registration Cutoff & Kick-off (OneShopAI Listing)	Tue, Nov 3, 2026	3:00 PM UTC (8:30 PM IST)	Listed on partner opportunity boards as the pre-kickoff registration target on lablab.ai[5].
Official Hackathon Kick-off Stream	Tue, Nov 3, 2026	4:00 PM – 5:00 PM UTC (8:00 AM – 9:00 AM PST)	Broadcast live; followed immediately by the Vultr Community & Discord Q&A session at 5:00 PM UTC (9:00 AM PST)[4].
Online Build Window	Nov 3 – Nov 8, 2026	Daily active window listed as 4:00 PM – 3:00 AM UTC (8:00 AM – 7:00 PM PST)	Continuous remote development on Vultr infrastructure; coordinated via the LabLab dashboard and LabLab Discord

			server[1].
On-Site Finale & Live Showcase (Salt Lake City)	Sun, Nov 8, 2026	Listed as 8:00 AM / 9:00 AM MST start (10-hour duration on Luma; 1:00 AM – 8:00 AM PST on Silicon Slopes)	Hosted at **Pierpont Place**, 163 W Pierpont Ave, Salt Lake City, UT 84101 by lablab, Mirdul Swarup, Mayank Debnath, and Joel Cox. Approval required; travel/lodging not covered[1].
Final Project Submission Deadline	Sun, Nov 8, 2026	Closes Nov 8, 2026	Official entries must be submitted via lablab.ai to qualify for judging and prizes[1].

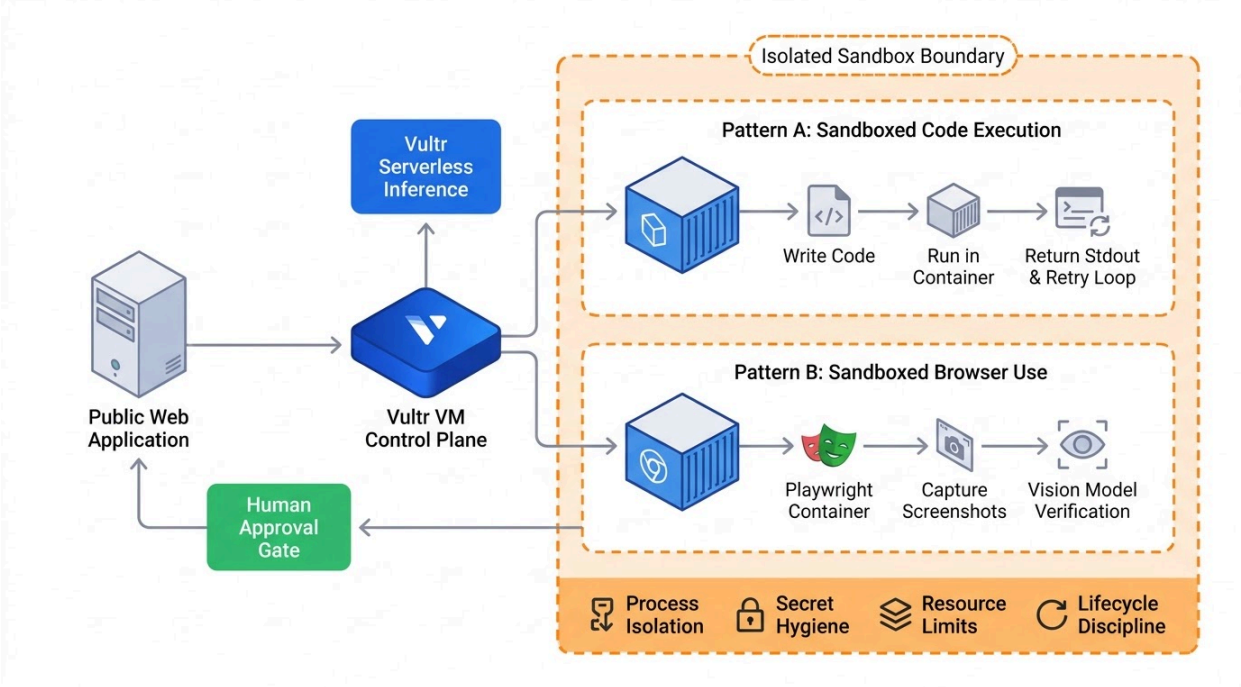
A notable technical quirk on the live LabLab dashboard (https://lablab.build/ai-hackathons/vultr-hackathon/live) is that the visual countdown widget displays 00 days : 00 hrs : 00 min : 00 sec during the active build week, even while the platform status badge explicitly reads **"Live · Submissions open"** and continues accepting new participants, teams, drafts, and final project submissions through the November 8 deadline[6]. Builders should not mistake the zeroed-out web timer for an early closure; submission intake remains active, as evidenced by the jump from 26 to 50 completed submissions and 14 to 20 active drafts on the live dashboard[6]. On the Vultr Community event page, participants also have a 3-step check-in mechanism (**Register → Attend +50 points** by entering an 8-character room code or scanning a QR code → **Participate**), which tracks community engagement alongside the primary LabLab submission portal[4].

The financial structure of the hackathon combines upfront infrastructure subsidies with tiered post-event payouts[1]. At the top of the funnel, every participant receives **$200 in free Vultr credits** upon registering a fresh Vultr account and redeeming the event gift code (while LabLab's generic site navigation bar displays a standard *"Join Challenge + Get $100 credits"* prompt, both the official LabLab event body and Vultr Community confirm the actual participant grant for Agent Rush is **$200**, with community tutorial videos also referencing tiered $200, $300, and $500 promotional credit structures across Vultr campaigns)[1]. At the conclusion of judging, the **$14,000 competition prize pool** is awarded to the top three placements: **1st Place** receives **$5,000 in cash plus $3,000 in Vultr credits** ($8,000 total value); **2nd Place** receives **$3,000 in cash plus $1,000 in Vultr credits** ($4,000 total value); and **3rd Place** receives **$1,000 in cash plus $1,000 in Vultr credits** ($2,000 total value)[1].

Neither the LabLab event page nor the Vultr Community portal splits the 1st, 2nd, and 3rd place cash and credit awards into separate pools per track; rather, teams choose one of the two challenge paths and compete on a unified leaderboard evaluated against four universal judging criteria[1]. Meanwhile, the **up to $1,000 bonus raffle** (one PlayStation 5 Standard, one pair of Apple AirPods 5 with Wireless Charging Case, and one Razer DeathAdder V3 HyperSpeed wireless gaming mouse) is restricted to three individual attendees who check in on-site in Salt Lake City on November 8[1].

Challenge Tracks: Architectural Mandates and Execution Patterns

Participants must select one of two challenge tracks and deploy a production-style web application where **Vultr Cloud Compute** serves as the central system of control and orchestration—not merely as a static frontend host[1]. While both tracks share the core requirement of a Vultr VM backend and a publicly accessible web URL, they diverge sharply in their execution mechanics, safety boundaries, and mandatory API dependencies[1].



Track 01: Blast Radius Zero (Containment-First Agent Execution)

Track 01: Blast Radius Zero is designed for systems engineers and AI developers building autonomous code-execution or browser-driving agents[1]. The guiding axiom of this track, stated in the Vultr Community brief, is that *"an agent that only chats is a demo, one that executes safely is a product"*[4]. Instead of returning static markdown suggestions, a Track 01 application must accept a natural-language objective, use **Vultr Serverless Inference** (https://api.vultrinference.com/v1) to plan and generate executable artifacts, dispatch those artifacts into an isolated sandbox environment, capture the real execution output or visual state, and return verifiable proof of work to the user[1].

Builders in Track 01 can implement **Pattern A (Sandboxed Code Execution)**, **Pattern B (Sandboxed Browser Use)**, or a hybrid of both[1]. Under **Pattern A**, the user submits a query or dataset, the model writes executable code (such as Python), the orchestrator dispatches that code into an isolated container or microVM, real output (stdout, stderr, or generated files) returns to the control plane, and the model explains the verified result—feeding runtime errors back into the loop for automated self-healing retries[1]. Official Pattern A reference archetypes include **CSV Analyst** (upload a spreadsheet, ask plain-English questions, and receive an executed chart plus the underlying code), **Prove-It Code Reviewer** (run a code snippet or pull request and its test suite inside the sandbox and report what happened), **Chart Anything** (natural language in, executed matplotlib or plotly image out), **File Fixer** (messy CSV or JSON in, cleaned file out with an executed before/after diff), and **Self-Healing Runner** (execute code, read errors, patch, and retry with the loop visible on screen)[4].

Under **Pattern B (Sandboxed Browser Use)**, the user provides a web goal, the model formulates a plan, a headless browser (such as Playwright) executes inside an isolated container, screenshots return to the orchestrator, a vision-capable model on Vultr Serverless Inference verifies each step, and a **human approval gate** intercepts execution prior to any final or committing actions[1]. Official Pattern B reference archetypes include **Comparison Shopper** (provide two or three product URLs and receive a screenshot-backed comparison table), **Form Filler** (structured data in, multi-step web form populated with a visual screenshot trail and an approve-before-submit gate), **Site QA Sweep** (crawl a web app to find broken flows and links with screenshots and reproduction steps), **Research with Receipts** (compile a brief where every claim links to a captured screenshot of its source), and **Page Watcher** (monitor listing or pricing pages, extract structured data, and diff against the previous day)[4].

Every Track 01 submission is evaluated on how rigorously it enforces **four containment focus areas**[1]. First, **process isolation** requires that untrusted code or browser sessions never execute inside the main web application runtime or touch the host OS; builders must isolate workloads using Docker containers, **Microsandbox** (KVM microVMs), **OpenSandbox**, **gVisor**, **E2B Sandboxes**, or throwaway Vultr Cloud Compute instances spun up and destroyed via the **Vultr API**[1]. Second, **secret hygiene** dictates that zero API keys or credentials reside inside the sandbox, keeping all privileged calls on the external Vultr VM control plane[1]. Third, **resource limits** require strict time and memory caps on every run (such as 256 MiB or 512 MiB memory allocations and per-command timeouts) to prevent host resource exhaustion[4]. Fourth, **lifecycle discipline** mandates resetting or destroying the sandbox environment after each task[1]. To prove these safeguards work in practice, Track 01 teams **must** include at least one live **"containment moment"** in their recorded demo video, showing the sandbox absorbing a destructive operation such as rm -rf, an infinite loop, or a hostile web page[1].

Track 02: Powering the Future of Work (Enterprise Workflow Orchestration)

Track 02: Powering the Future of Work targets builders creating multi-step, enterprise-focused AI solutions that improve daily team operations[1]. Here, Vultr is looking for realistic business workflows—either agentic, rule-based, or hybrid—deployed as a

production-style web application where a Vultr VM backend powers planning, workflows, and operations[1]. Rather than a one-off local demo, submissions must demonstrate platform thinking across five highlighted enterprise focus areas: **Sales Operations** (qualifying leads, researching prospects, updating CRMs, drafting outreach, and following up); **Finance & Accounting** (processing invoices, reconciling transactions, flagging anomalies, and managing payment follow-ups); **HR & Recruitment** (screening candidates, scheduling interviews, coordinating onboarding, and tracking employee workflows); **Customer Support** (resolving tickets across systems, investigating issues, taking actions, and escalating to humans when needed); and **Procurement** (monitoring purchasing needs, comparing vendors, requesting quotes, tracking orders, and following up on delays)[1].

Requirement / Dimension	Track 01: Blast Radius Zero	Track 02: Powering the Future of Work
Primary Engineering Goal	Safe, isolated execution of untrusted agent-generated code or headless browser actions with verifiable outputs[1].	End-to-end coordination of realistic, multi-step enterprise workflows with production-grade user flows[1].
Vultr Cloud Compute (VM Backend)	**Mandatory** — Must act as the central control and orchestration layer[1].	**Mandatory** — Must act as the central backend powering planning, workflows, and operations[1].
Web Application Hosted on Vultr	**Mandatory** — Publicly accessible demo URL required[1].	**Mandatory** — Publicly accessible web browser experience required[1].
Vultr Serverless Inference API	**Mandatory** — All agent LLM calls (including vision verification) must route through https://api.vultrinference.com/v1[1].	**Optional** — Recommended for agentic steps, but rule-based or hybrid LLM stacks are permitted[1].
Isolated Agent Sandboxing	**Mandatory** — Must use Docker, Microsandbox, OpenSandbox, gVisor, E2B, or throwaway Vultr API instances[1].	**Optional** — Useful if workflows execute dynamic scripts, but not a baseline track requirement[1].

Deployment Accelerators	Optional use of **Vultr API** to spin up and destroy throwaway instances per task[1].	Recommended use of **Supabase on Vultr** (BaaS) and **Coolify on Vultr** (PaaS)[1].
GPU Availability	**Not Available** — Vultr GPUs are excluded from the hackathon environment[1].	**Not Available** — Vultr GPUs are excluded from the hackathon environment[1].
Mandatory Video Proof	Must include a live **"Containment Moment"** (e.g., sandbox absorbing rm -rf, infinite loop, or hostile site)[1].	Must demonstrate a cohesive, platform-style multi-step enterprise user flow rather than a local demo[1].

Core Technical Stack and Implementation Playbook on Vultr

To accelerate development during the November 3–8 build window, Vultr's developer relations team—including Junior DevRel Sanskriti Harmukh—provided documentation and video walkthroughs covering four foundational tasks: claiming the $200 hackathon credit grant, integrating **Vultr Serverless Inference**, configuring hardware-isolated **Microsandbox** environments on Vultr Cloud Compute, and deploying web stacks via **Supabase** and **Coolify** Marketplace Applications[1].

1. Claiming the $200 Free Vultr Hackathon Credits

Every participant can provision Vultr Cloud Compute instances, Managed Databases, Kubernetes clusters, and Serverless Inference subscriptions using the $200 promotional grant by completing the following sequence[1]:

1. **Create a Fresh Vultr Account:** Navigate to the Vultr registration portal and sign up using an email address that has **not** previously been used on Vultr[1].
2. **Verify Email & Account Identity:** Complete the email verification link sent to your inbox[1].
3. **Redeem the Hackathon Gift Code:** In the Vultr Customer Portal (my.vultr.com), navigate to **Billing > Gift Code** (or the dedicated Hackathon Promotion page linked on LabLab) and apply the hackathon gift code (builders on the live dashboard report redeeming coupon code VULTRAGENTRUSH26)[1].

2. Integrating Vultr Serverless Inference & the Model Catalog SDK

For Track 01 teams (where it is mandatory) and Track 02 teams (where it is optional), **Vultr Serverless Inference** provides an auto-scaling, OpenAI-compatible inference service at the base URL https://api.vultrinference.com/v1[1]. To activate a subscription via the Vultr Console, navigate to **Compute > Serverless Inference**, click **Create Inference**, enter a label for the subscription, review the usage-based pricing details and select **"I acknowledge the**

usage-based cost", and click **Create Inference** until the status changes to **Active** (subscriptions can also be provisioned via the Vultr API, Vultr CLI, and Terraform)[16].
Beyond standard chat completions, Vultr Serverless Inference exposes a multi-modal REST surface across ten model categories: **Chat and reasoning**, **Images**, **Video**, **Music**, **Speech**, **Transcription**, **Alignment**, **Embeddings**, **Rerank**, and **Decisions**[16]:

Functional Category	**Vultr Serverless Inference REST Endpoints (https://api.vultrinference.com)**	**Primary Hackathon Use Case**
Chat, Messages & Native RAG	POST /v1/chat/completions, POST /v1/chat/completions/RAG, POST /v1/messages, POST /v1/responses, GET /v1/responses/{id}, DEL /v1/responses/{id}	Core agent planning, tool-calling loops, vision screenshot verification, and retrieval-augmented generation[4].
Autonomous Decisions	POST /v1/systemone	Low-latency decision applications and policy routing[16].
Managed Vector Store & Files	GET/POST /v1/vector_store, GET/DEL/PATCH /v1/vector_store/{id}, POST /v1/vector_store/{id}/search, GET/POST/DEL/PATCH /v1/vector_store/{id}/items/{itemid}, GET/POST/DEL /v1/vector_store/{id}/files/{fileid}, GET/POST/DEL /v1/files/{id}	Hosting enterprise knowledge bases, SOPs, and document corpora for Track 02 workflows[17].
Embeddings & Reranking	POST /v1/embeddings, GET /v1/embeddings/models, POST /v1/rerank	Semantic search and two-stage retrieval reranking[16].
Audio & Voice Pipeline	POST /v1/audio/speech, POST /v1/audio/transcriptions,	Speech generation (text-to-speech), audio transcription, and

	POST /v1/audio/alignments, GET /v1/audio/models, GET /v1/audio/voices	alignment[16].
Image, Video & Music Generation	POST /v1/images, POST /v1/images/generations, POST /v1/images/edits, POST /v1/videos, POST /v1/videos/edits, POST /v1/music, GET /v1/music/{id}/stream	Generating visual, video, or music assets for creative and marketing workflows[16].
Catalog, Telemetry & Webhooks	GET /v1/models, GET /v1/models/all, GET /v1/models/count, GET /v1/model/{author}/{slug}, GET /v1/usage, GET /v1/health, GET/POST /v1/webhooks/secret	Live model discovery, usage tracking, health checks, and webhook secret rotation[17].

To query available models dynamically and filter for those capable of driving a coding or tool-calling agent, Vultr provides official zero-dependency client libraries for **Python 3.11+** (pip install vultr-model-catalog) and **TypeScript / Node 20+ / Bun** (npm install @vultr/model-catalog)[17]. Both libraries fetch GET /v1/models (which requires **no API key**), flatten OpenRouter provider Model Documents into a single normalized object, and support offline disk caching (cache_path=".vultr-catalog.json") so applications can fall back to the last known good catalog if a network fetch fails[17]. Crucially, the catalog provides a built-in catalog.agent_models property (or isAgentModel filter in TypeScript) that filters for chat models that are ready to serve traffic (is_ready), support function tool calling (tools), and have a known context_window, while exposing exact string-based USD token pricing (price_per_million()) and reasoning.supported_efforts[17]:

```
Python
from vultr_model_catalog import load_catalog

# Fetch live Vultr model catalog with 5-minute local disk cache (no API key needed)
catalog = load_catalog(
    base_url="https://api.vultrinference.com/v1"
```

```
cache_path=".vultr-catalog.json"
max_age=300

for model in catalog.agent_models:
price = model.price_per_million()
reasoning = model.reasoning
efforts = " ".join(reasoning.supported_efforts or []) if reasoning else "-"
print(model.id, model.context_window, price["prompt"], price["completion"], efforts)
```

3. Hardware-Isolated Agent Sandboxing with Microsandbox on Vultr VX1

While standard Docker containers are widely used by Track 01 participants, traditional containers share the host Linux kernel—meaning a kernel bug or container escape could still threaten the host VM[2]. To achieve stronger kernel-level isolation, Vultr's official agent sandboxing guide demonstrates **Microsandbox (msb)** deployed on a **Vultr VX1 Cloud Compute** instance running **Ubuntu 24.04**[15]. Microsandbox uses libkrun over the Kernel-based Virtual Machine (/dev/kvm) to boot standard Open Container Initiative (OCI) images as lightweight **microVMs**, each running its own separate Linux kernel under non-root kvm group permissions[15]. Host-to-sandbox communication and SSH access (msb exec, msb ssh, msb copy) operate over internal vsock channels rather than exposed network ports, eliminating the need for an in-guest SSH daemon[15]. Furthermore, while sandboxes have outbound internet access by default, peer-to-peer network traffic between separate microVMs on the same host (sandbox01 to sandbox02) is blocked by default without requiring host firewall rules, and every microVM enforces explicit CPU and memory caps (--cpus 1 --memory 256M)[15].

Builders can deploy a complete Microsandbox and Vultr Serverless Inference agent runtime on an Ubuntu 24.04 Vultr VX1 instance in five steps[15]:

1. **Enable Non-Root KVM Access:** Add your non-root sudo user to the kvm group (sudo usermod -aG kvm $USER && newgrp kvm) and verify that /dev/kvm exists and belongs to the kvm group (id && ls -l /dev/kvm)[15].
2. **Install the Microsandbox Runtime:** Run curl -fsSL https://install.microsandbox.dev | sh, log out and back in over SSH for PATH updates to take effect, and verify host readiness via msb --version and msb doctor (on Ubuntu's default ext4 filesystem, an informational warning notes that msb falls back from reflink clones to sparse copies for sandbox roots)[15].
3. **Provision & Constrain Sandboxes via CLI:** Run ephemeral commands (msb run debian -- echo "hello from microsandbox") or create detached, resource-capped microVMs (msb create --name sandbox01 --cpus 1 --memory 256M debian), inspect live resource usage via msb status sandbox01, msb metrics sandbox01, and msb logs sandbox01 --tail 5, and destroy finished instances via msb stop and msb remove[15].
4. **Integrate Programmatically via the Python SDK:** Install python3-venv and pip install

microsandbox inside a virtual environment to spawn disposable microVMs (async with await Sandbox.create("py-demo", image="python", cpus=1, memory=512, replace=True) as sandbox:) directly from a Python backend (SDKs are also available for TypeScript, Rust, and Go)[15].

5. **Connect Coding Agents via Model Context Protocol (microsandbox-mcp):** Install Node.js 22 or later (curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash - && sudo apt install -y nodejs) and configure an MCP-compatible coding agent such as **OpenCode** (opencode.json) using @ai-sdk/openai-compatible pointed at https://api.vultrinference.com/v1 alongside "command": ["npx", "-y", "microsandbox-mcp"], enabling the agent to create and run code inside isolated microVMs on demand[15].

4. Rapid Web App Deployment: Supabase and Coolify Marketplace Stacks

Because every submission must provide a live, publicly accessible product URL hosted on a Vultr VM, the organizers highlight two one-click **Vultr Marketplace Applications** to streamline full-stack deployment[1].

For teams building **Next.js + Supabase** applications—particularly Track 02 enterprise workflows requiring PostgreSQL, Row-Level Security (RLS), user authentication, real-time subscriptions, and over 50 pre-configured extensions like uuid-ossp and pgcrypto—builders can provision a Vultr Cloud Compute instance using the **Supabase** Marketplace image (image_id set to supabase via API/Terraform or vultr-cli instance create --image "supabase")[18]. After connecting via SSH, retrieve the generated SUPABASE_JWT_SECRET from the Vultr Console App Instructions, use Node's jsonwebtoken package to sign fresh HS256 tokens for ANON_KEY and SERVICE_ROLE_KEY, update /home/supabase/supabase/docker/.env, and recreate the containers (docker compose down && docker compose up -d)[18]. Finally, configure **Nginx** on port 80/443 with a split reverse-proxy configuration that routes root requests (/) to the Next.js frontend at http://127.0.0.1:3000 and /supabase/ requests to the Supabase Kong API gateway at http://127.0.0.1:8000/[18].

For teams deploying multi-container docker-compose.yml or Dockerfile applications—including projects scaffolded with Cursor, Claude Code, Lovable, or Bolt.new—builders can deploy the **Coolify** Marketplace Application on a Vultr VM with at least **2 CPU cores and 4 GB RAM**[24]. After creating an admin account at http://SERVER_IP:8000, link your GitHub account via OAuth under **Sources** and create a project[21]. Coolify automatically detects Docker Compose build packs, generates an immediate temporary test domain (RANDOM_ID.SERVER_IP.sslip.io), configures **Traefik** reverse-proxy routing, provisions Let's Encrypt TLS certificates in 3–5 minutes, and provides a GitHub webhook URL for automatic rebuilds on every git push origin main[24]. Teams can also connect a **Vultr Object Storage** bucket under Coolify's **S3 Storages** settings to automate scheduled daily backups of PostgreSQL 16 databases[21].

Live Competition Landscape, Tooling Telemetry, and Benchmark Projects

Comparing the two live dashboard snapshots captured during the build window reveals rapid growth in both builder participation and project submissions[6]. Between the earlier and latest telemetry captures on lablab.build/ai-hackathons/vultr-hackathon/live, total registered participants rose from **2,440 to 2,536** (with +97 joining on the latest day), formed teams expanded from **670 to 714** (+50 on the latest day, working out to roughly 3.55 participants per team across the cohort alongside numerous solo builders), completed submissions nearly doubled from **26 to 50** (+29 on the latest day), and active drafts rose from **14 to 20**[2].
The live **Tools Being Used This Round** leaderboard also captures a clear shift in model and tooling selection[6]. In the earlier 26-submission snapshot, the most frequently tagged tools were **VULTR (30)**, **AgentOps (6)**, **AI/ML API (5)**, **rest api (4)**, and a four-way tie at **3** among **Antigravity**, **DeepSeek V3**, **Claude Code**, and **OpenAI**[6]. By the 50-submission snapshot, **VULTR** rose to **59** tags and **Qwen** surged to **#2 overall with 37 tags**—demonstrating strong adoption of Qwen models on Vultr Serverless Inference—followed by **AgentOps (7)**, **AI/ML API (5)**, **DeepSeek V3 (5)**, **OpenAI (5)**, **rest api (4)**, **Claude Code (4)**, **Codex (4)**, **NVIDIA (3)**, and **Antigravity (3)**[6].

VULTR: AGENT RUSH HACKATHON · COMPETITIVE ARCHITECTURE MATRIX

2,536 PARTICIPANTS · 714 TEAMS · 50 SUBMISSIONS

Track 01 Sandboxed Code Runners Dominate Early Submissions While Specialized Control Agents Lead Niche Verticals

Track 1: Pattern A Code (7) | Track 1: Pattern B Browser (2) | Track 2: Enterprise Workflows (2) | Other Live Finalists (3)

Showing 14 of 14 verified builds

All Projects (14) | Track 1: Pattern A Code (7) | Track 1: Pattern B Browser (2) | Track 2: Enterprise Workflows (2) | Other Live Finalists (3)

Filter by keyword (e.g., Docker, MCP, Vision, Invoice, ROCm, SHA)...

Try: Docker MCP Vision Invoice

PROJECT & BUILDER	TRACK / PATTERN	SANDBOX & VULTR STACK	VERIFICATION & SAFETY GATE	VOTES & STATUS
SAAS Fashion Garment Marketing Digital Control SuperInfra Trinity V32-2 / Tuan Cin ★ Featured on Vultr Community	Track 2: SME ERP/SCM Digital Control Tower	Vultr Serverless Inference + AMD ROCm (MI300X) for Batik trend forecasting & SCM V4 Vultr Inference · AMD MI300X · ROCm	Yard/Kodi inventory control, Sritex invoice verification, OEE 95% & KPI monitoring	2-3 votes #1 Live Dashboard
Detonation Chamber Builders Nepal	Track 1: Containment Sandboxed Runtime	Vultr isolated detonation sandbox environment Vultr Sandbox · Containment	Blast-radius containment chamber for untrusted agent execution	2 votes Tied #1 Live Vote
QuantGate WataruAkis (Solo)	Track 1: Pattern A Code Quantization Verifier	Disposable Python containers on Vultr running generated verification kernels Python Kernels · Disposable Containers	Tests model quantization, detects exact data overlap & blocks releases exceeding limits	1 vote Live Finalist
YAI Guard — Prove-It Code Runner YAI Guard	Track 1: Pattern A Code Prove-It Code Reviewer	Vultr isolated code execution sandbox Code Runner · Vultr VM	Executes code & test suites in sandbox with verifiable execution proof	1 vote Live Finalist
JARVIS ProofOps JARVIS ProofOps	Track 1: Pattern A Code Proof-Backed Ops Agent	Sandboxed operational execution runtime on Vultr ProofOps · Vultr Sandbox	Cryptographic / verifiable operational run receipts and containment checks	1 vote Live Finalist
AgentRush — Blast Radius Zero Kosoyy / Lu guowei (Solo) ★ Featured on Vultr Community	Track 1: Pattern A Code Sandboxed Agent Runtime	Fresh throwaway, network-less Docker sandbox on Vultr Docker · Network-less · Vultr	Verifiable run records: command details, SHA output hashes & container destruction receipts	0 votes Live Finalist
ProveRun peterzhou / Peter Zhou (Solo)	Track 1: Pattern A Code Blast Radius Zero Runner	Vultr Serverless Inference writes Python → runs in disposable Docker sandbox Vultr Inference · Docker · Python	Captures real stdout + LLM explanation from isolated throwaway container	Active Build Documented Team
Airlock: Blast-Radius-Zero Code Agent AmberNeel / Abdul AmberNeel	Track 1: Pattern A Code Self-Healing Runner	Network-less, zero-secret disposable code sandbox on Vultr Zero-Secret · Network-less · Self-Healing	Policy gate with human approval, self-healing error loop, independent verification & run receipt	Active Build Documented Team
PayablesPilot ledger-scout1 / Florin Chiroaica	Track 1 Hybrid / Track 2 Finance AP Control Agent	Safely sandboxed AI agent hosted on Vultr with strict execution limits Accounts Payable · Invoice Match · Sandboxed	3-way invoice match, duplicate prevention, supplier bank-detail control + human approval gate	Active Build Documented Team
Ghostfox Zero: Browser Agent autokeren / Ajat Sudrajat (Solo)	Track 1: Pattern B Browser MCP Browser Sandbox	Vultr Serverless Inference + disposable custom Firefox fork & Rust MCP container Firefox Fork · Rust MCP · AI Vision	Semantic accessibility reading, step-by-step AI vision verification & human approval gate	Active Build Documented Team
SiteDoctor: Sandboxed AI Website Doctor SiteDoctor	Track 1: Pattern B Browser Site QA Sweep	Sandboxed browser automation & QA inspection runtime on Vultr Browser QA · Sandbox	Automated website health sweep & isolated defect detection	0 votes Live Finalist
AI-Powered Recruitment Platform AI_Builders	Track 2: Enterprise HR Hiring Control Workflow	Vultr-hosted recruitment orchestration & analytics engine HR Workflow · Recruitment Analytics	Automated screening & interview scheduling with human approval gates at critical decisions	Active Build Documented Team
VOIDLOCK TECH FORGE	Track 1: Containment Sandboxed Agent	Vultr containment-first isolated runtime VOIDLOCK · Vultr Sandbox	Strict sandbox boundary enforcement for agent tasks	0 votes Live Finalist
SRE Incident Agent — Blast Radius Zero SRE Incident Agent	Track 1: Containment Incident Triage Agent	Blast-Radius-Zero SRE diagnostic sandbox on Vultr SRE Ops · Blast Radius Zero	Contained incident investigation without host runtime exposure	0 votes Live Finalist

Analysis of documented team builds and top-ranked live dashboard entries shows a strong concentration around **Track 01 Pattern A** (disposable Docker code runners with cryptographic receipts) and **Pattern B** (custom Firefox/MCP browser sandboxes), alongside targeted **Track 02** enterprise control towers in accounts payable, recruitment, and SME supply chains.

Data sources: lablab.ai Live Hackathon Dashboard, Vultr Community Showcase, and documented team workspaces (AgentRush, SuperInfra Trinity, QuantGate, ProveRun, Airlock, Ghostfox Zero, PayablesPilot, AI_Builders).

Deep-Dive Analysis of Notable Active Builds and Architectural Patterns

Inspecting the technical designs of teams currently building in the hackathon—alongside the two projects already featured on the official **Vultr Community Showcase**—reveals three distinct architectural patterns across the field[2].

The first major pattern is **verifiable execution receipts and cryptographic audit trails in Track 01 (Pattern A)**. Rather than merely streaming raw stdout from a container, several Pattern A builders generate structured execution receipts for every sandbox run[20]. **AgentRush — Blast Radius Zero** (built by solo developer Kosoyy / Lu guowei, hosted at github.com/kosoyyagent/agentrush, and featured on the Vultr Community page) plans tasks, fetches web pages, and executes all generated code inside a fresh, network-less disposable Docker sandbox on Vultr, returning verifiable run records with command details, output hashes, and container destruction status[4]. Similarly, **Airlock: Blast-Radius-Zero Code Agent** (by solo builder Abdul AmberNeel) pairs a disposable, zero-secret, network-less sandbox with a pre-execution human policy gate, automated self-healing from real runtime errors, independent verification, and a receipt for every run[29]. Meanwhile, **ProveRun** (by Peter Zhou) implements a focused Pattern A pipeline (ask → Vultr Inference writes Python → disposable Docker sandbox runs it → real stdout + explanation)[2], and **QuantGate** (by solo Systems & AI Engineer WataruAkis) applies Pattern A to ML release verification—using Vultr Serverless Inference to generate and run verified Python kernels in disposable containers that test model quantization, detect exact data overlap, and block model releases that exceed storage or accuracy limits[6].

The second pattern centers on **custom browser engines with vision verification and MCP endpoints in Track 01 (Pattern B)**. While the official prompt suggests Playwright in a container, solo builder Ajat Sudrajat (team autokeren) engineered **Ghostfox Zero: Browser Agent**, which pairs a custom **Firefox fork** with a **Rust Model Context Protocol (MCP) runtime** inside a disposable container[30]. Ghostfox Zero uses **Vultr Serverless Inference** to read live web pages via semantic accessibility, executes one action at a time inside the disposable Ghostfox container, visually verifies every step using AI vision, enforces a human approval gate before any committing action, and exposes an MCP endpoint for integration with external AI agents[30]. Other Pattern B contenders visible on the live leaderboard include **SiteDoctor** (a sandboxed AI website doctor)[6].

The third pattern involves **domain-specific enterprise control towers and policy checkpoints in Track 02 and hybrid builds**. In the Finance & Accounting vertical, **PayablesPilot** (by solo builder Florin Chiroaica / ledger-scout1) operates as an accounts-payable control agent on Vultr that enforces three pre-payment verification checks before any supplier invoice can be paid: a **three-way match** (verifying invoices against purchase orders and delivery receipts), **duplicate invoice prevention**, and **supplier bank-detail control** to prevent payment fraud, backed by strict execution limits and human approval for sensitive actions[31]. In the HR vertical, team **AI_Builders** developed an **AI-Powered Recruitment Platform** that coordinates candidate screening, interview invitations, applicant

status tracking, and recruitment analytics with human approval checkpoints at critical hiring decisions[32]. Meanwhile, a top-voted community entry across both snapshots—**SAAS Fashion Garment Marketing Digital Control** by **SuperInfra Trinity V32-2** (Tuan Cin, also featured on the Vultr Community showcase)—targets 10M+ Indonesian fashion SMEs with an end-to-end supply chain control tower spanning AI Batik motif trend forecasting via the Vultr Serverless Inference API, Yard/Kodi control, Sritex invoicing, 95% OEE tracking, and digital marketing[4]. (Note: Although SuperInfra Trinity V32-2's team profile mentions AMD MI300X / ROCm optimization, builders should remember that Vultr Cloud GPUs are not available as part of the Agent Rush hackathon infrastructure[1].)

Lessons from Precedent: What Vultr × LabLab Judges Reward

To understand what Vultr and LabLab prioritize in winning projects, participants can examine the official Vultr Blog recap of the **February 2026 Vultr × LabLab "Launch and Fund Your Own Startup" AI and Robotics Hackathon**, whose three winning teams presented live at **NVIDIA GTC 2026** at Vultr booth 1631[33]. In that competition, **Sovereign Robotics Ops** won in the manufacturing and energy category by building a FastAPI, PostgreSQL, Docker, and Next.js/Tailwind **AI Governance Layer** on Vultr infrastructure that acted as a safety and compliance checkpoint between Gemini Robotics AI planning and physical execution—evaluating human proximity, speed thresholds, and geofencing rules before deciding whether to execute, modify, or stop a plan[33]. A second winner, **DroneOS**, built a centralized cloud-to-drone operations center on a **Vultr-hosted VPS** with an **OpenClaw** AI dispatch layer, VPN-connected incident intake, and a real-time React dashboard that optimized fleet dispatch while retaining optional human override[33]. The third winner, **CarphaCom**, built an event-driven microservices platform on Vultr connecting online e-commerce orders to real-time warehouse execution[33]. Across these winners, Vultr consistently rewarded architectures that used **Vultr infrastructure as a deterministic governance, safety, and orchestration control layer** with clear human-in-the-loop oversight[1].

Judging Framework, Rules, and Mandatory Submission Checklist

While individual judge names are not listed on the LabLab or Vultr Community pages, all eligible entries are evaluated across **four official judging criteria**[1]. **Application of Technology** evaluates how effectively the chosen models and Vultr infrastructure components are integrated into the solution[1]. **Presentation** assesses the clarity and effectiveness of the recorded demo video, slide deck, and architecture explanation[1]. **Business Value** measures the practical impact and fit within real-world business or operational areas[1]. Finally, **Originality** evaluates the uniqueness and creativity of the solution and its demonstrated agent behaviors[1].

Complete Pre-Submission Verification Checklist

Before the **Sunday, November 8, 2026** deadline closes, teams should verify that every required administrative, technical, and documentation deliverable is complete[1]:

1. **Dual Platform Registration:** Ensure every team member (1 to 6 people) is registered for

the event on lablab.ai **and** has joined the official **LabLab Discord server** (discord.gg/lablabai)[1]. If attending the Salt Lake City finale on November 8 at Pierpont Place, confirm that your on-site participation has been approved[3].
2. **Vultr VM Control Plane & Live Public URL:** Confirm your backend orchestrator and web application are deployed on a **Vultr Cloud Compute VM** and accessible via a public demo URL offering a real product-style experience[1].
3. **Vultr Serverless Inference Integration:** For Track 01 (where it is required) and Track 02 (if using Vultr LLMs), verify that agent LLM calls route through https://api.vultrinference.com/v1 and that your Vultr Serverless Inference subscription remains active[1].
4. **Sandbox Containment Verification (Track 01 Mandatory):** Confirm that untrusted code or browser sessions run strictly inside isolated containers or microVMs (Docker, Microsandbox, gVisor, OpenSandbox, E2B, or throwaway Vultr API instances) with process isolation, zero secrets inside the sandbox, strict time and memory caps, and environment reset or destruction after each task[1].
5. **Recorded Demo Video with "Containment Moment":** Record a demo video demonstrating the live web application[1]. If submitting to **Track 01**, the video **must** include at least one **containment moment**—such as the sandbox absorbing rm -rf, terminating an infinite loop, or isolating a hostile web page[1].
6. **Public MIT-Compliant GitHub Repository & Vultr README:** Publish your original source code under an **MIT-compliant license** in a public GitHub repository[1]. Ensure the README.md clearly documents: (a) the Vultr services and infrastructure used, (b) the backend and deployment architecture showing how Vultr powers the solution, and (c) the agent's use case and execution of real tasks[1].
7. **Official LabLab Submission Portal Completion:** Submit the complete entry on lablab.ai—including project title, short description, long description, technology and category tags, cover image, slide presentation, video presentation, public GitHub URL, demo application platform, and public application URL[1].
8. **Optional Vultr Community Showcase:** After completing your mandatory lablab.ai submission, optionally share your project on the **Vultr Community** portal (community.vultr.com/e/vultr-agent-rush-hackathon) for additional visibility across the Vultr developer ecosystem[1].

Works cited

1. Vultr: Agent Rush Hackathon - LabLab.ai, https://lablab.build/ai-hackathons/vultr-hackathon
2. Vultr: Agent Rush Hackathon team: peterzhou - LabLab, https://lablab.build/ai-hackathons/vultr-hackathon/peterzhou
3. AI Hackathons | lablab.build, https://lablab.build/ai-hackathons
4. Vultr: Agent Rush Hackathon, https://community.vultr.com/e/vultr-agent-rush-hackathon

5. Vultr Agent Rush Hackathon - OneShopAI, https://community.oneshopai.com/opportunities/vultr-agent-rush-hackathon-930aa01d-3182-4aa7-8ce2-1500c2a963f4
6. Vultr: Agent Rush Hackathon - LabLab.ai, https://lablab.build/ai-hackathons/vultr-hackathon/live
7. AI Events - Cerebral Valley, https://cerebralvalley.ai/events
8. AI Events in Salt Lake City - Luma, https://luma.com/discover/salt-lake-city/ai
9. AI Events, Conferences & Hackathons — Updated Weekly, https://www.agenticbrew.ai/events
10. 10X Growth Strategies · Events Calendar - Luma, https://luma.com/10xgrowthstrategies
11. Vultr Agent Rush Hackathon: Salt Lake City On-site Day, https://www.siliconslopes.com/c/ai-events/vultr-agent-rush-hackathon-salt-lake-city-on-site-day
12. Vultr: Agent Rush Hackathon — Dates & Registration, https://www.hackathonradar.com/database/hackathon/vultr-agent-rush-hackathon-2026-c391
13. Vultr: Agent Rush Hackathon - Dapp.ing, https://dapp.ing/hackathons/vultr-agent-rush-hackathon-2026
14. How to claim Vultr Agent Rush Hackathon $200, $300 ... - YouTube, https://www.youtube.com/watch?v=N00a-HElH0I
15. Agent Sandboxing on Vultr Cloud Compute with Microsandbox, https://docs.vultr.com/how-to-set-up-agent-sandboxing-on-vultr-cloud-compute
16. https://docs.vultr.com/products/serverless/inference/provisioning
17. Model catalog libraries - Vultr Serverless Inference, https://api.vultrinference.com/docs/integrations/model-catalog
18. Deploy Next.js Apps with Vultr Supabase Marketplace App, https://docs.vultr.com/how-to-deploy-a-nextjs-application-with-vultr-supabase-marketplace-app
19. Vultr: Agent Rush Hackathon team: SuperInfra Trinity V32-2 - LabLab, https://lablab.build/ai-hackathons/vultr-hackathon/aichaliveret
20. Vultr: Agent Rush Hackathon team: AgentRush - Kosoyy - LabLab, https://lablab.build/ai-hackathons/vultr-hackathon/agentrush-kosoyy
21. How to Use Vultr's Coolify Marketplace Application, https://docs.vultr.com/how-to-use-vultr-s-coolify-marketplace-application
22. How to Use Vultr's Supabase Marketplace Application Guide, https://docs.vultr.com/how-to-use-vultr-s-supabase-marketplace-application
23. How to Install Supabase on Ubuntu 20.04 Complete Guide | Vultr Docs, https://docs.vultr.com/how-to-install-supabase-on-ubuntu-20-04
24. How to Deploy Lovable Projects on Vultr Using Coolify, https://docs.vultr.com/how-to-deploy-lovable-projects-on-vultr-using-coolify
25. How to Deploy Claude Code Projects on Vultr Using Coolify, https://docs.vultr.com/how-to-deploy-claude-code-projects-on-vultr-using-coolify

26. How to Deploy Bolt.new Projects on Vultr Using Coolify, https://docs.vultr.com/how-to-deploy-bolt-new-projects-on-vultr-using-coolify
27. How to Deploy Cursor Projects on Vultr Using Coolify, https://docs.vultr.com/how-to-deploy-cursor-projects-on-vultr-using-coolify
28. Vultr: Agent Rush Hackathon team: WataruAkis - LabLab, https://lablab.build/ai-hackathons/vultr-hackathon/wataruakis
29. Vultr: Agent Rush Hackathon team: AmberNeel - LabLab, https://lablab.build/ai-hackathons/vultr-hackathon/amberneel
30. Vultr: Agent Rush Hackathon team: autokeren - LabLab, https://lablab.build/ai-hackathons/vultr-hackathon/autokeren
31. Vultr: Agent Rush Hackathon team: ledger-scout1 - LabLab, https://lablab.build/ai-hackathons/vultr-hackathon/ledger-scout1
32. Vultr: Agent Rush Hackathon team: AI_Builders - LabLab.ai, https://lablab.build/ai-hackathons/vultr-hackathon/aibuilders
33. Inside the Winning Use Cases from Our Latest Hackathon - Vultr Blogs, https://blogs.vultr.com/robotics-use-case-hackathon
34. Vibe Jams: AI & Vibe Coding Hackathons | VCL Events, https://vibecodinglist.com/vibe-jams
35. Vultr: Agent Rush Hackathon... - VibeCodingList Vibe Jams, https://vibecodinglist.com/vibe-jams/vultr-agent-rush-2026-ai-agent-hackathon